

UE24CS341A – Software Engineering

Lab 4: Vibe Coding

Fixing and extending a terminal 2048 game with ChatGPT

Name	Apurv Kumar Singh
SRN	PES1UG24CS077
Class / Section	5B
Course	UE24CS341A Software Engineering
Instructor	Dr. Sapna V M
Assigned repository	#14 – SETAPESU26/14_2048 (terminal 2048, Python)
AI tool used	ChatGPT (4 prompts, one per task)
My repository	https://github.com/ofsadug/Micro-Lending-Peer-Credit-Risk-Assessor (folder Lab-4/)

1. What this lab was about

The idea behind this lab was "vibe coding": you get a small project that has been broken on purpose, and you fix it and finish its missing features by working with an AI assistant instead of writing everything yourself. We could only use 3–4 prompts, so each one had to be clear and to the point. Every task had to go in its own commit, and we had to record short before and after videos and export the whole chat.

I was given repository **#14 – 14_2048**, a terminal version of 2048 written in Python. It has three files: `main.py` (starts the game), `board.py` (the grid, sliding and merging) and `game.py` (the game loop, input and display). The README listed four tasks. I used **ChatGPT** and gave it **one prompt per task, so 4 prompts in total**. After every reply I copied the code in, ran it myself, checked it against the README, and only committed it once it worked.

Everything is in the Lab-4/ folder of my own repository. I didn't open a pull request to SETAPESU26.

Deliverables at a glance

Deliverable	Where to find it
Fixed and extended code	Lab-4/board.py, Lab-4/game.py
One commit per task	dfb760d, 3bf024a, 5586c48, 33dfc00 (Section 8)
Prompts (4)	Sections 3–6, each with a screenshot of the prompt
Before video	PES1UG24CS077 Lab4 Before.mp4
After video	PES1UG24CS077 Lab4 After.mp4
Chat history	ChatGPT share link + Lab-4/PES1UG24CS077_Lab4_Chat_History.pdf
README	Root README has a Lab 4 entry

2. First look: what was broken

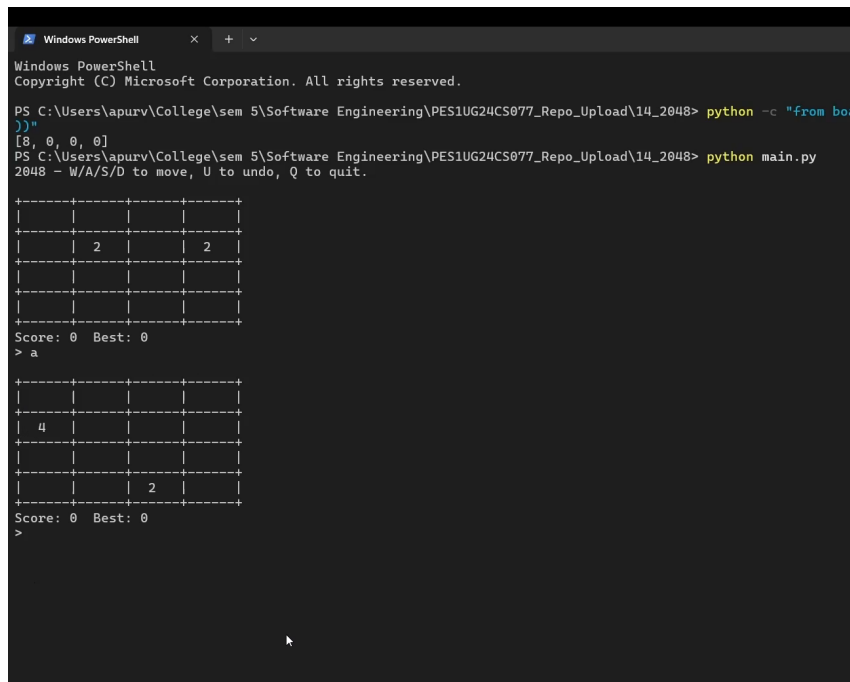
Before asking the AI anything, I ran the starter code (commit 249cf2d) to see what was wrong. This is what I found:

#	What I saw	Where it comes from
1	Tiles merged twice in one move. <code>[2, 2, 4, 0]</code> became <code>[8, 0, 0, 0]</code> instead of <code>[4, 4, 0, 0]</code> .	<code>board.py</code> – <code>slide_line</code>
2	The score stayed at 0 no matter how many tiles merged.	<code>board.py</code> – <code>slide_line</code>
3	Typing nothing, or something like "as", was silently accepted because <code>key not in "wasd"</code> checks for a substring.	<code>game.py</code> – <code>run</code>
4	Pressing U only said "Undo is not implemented yet."	<code>game.py</code> – <code>run</code>
5	No message after a move to say what happened.	<code>game.py</code>

The bug that causes most of the trouble is in the starter `slide_line`:

```
def slide_line(line):
    values = [x for x in line if x]
    result = []
    for value in values:
        if result and result[-1] == value:
            result[-1] *= 2 # intentional double-merge bug
        else:
            result.append(value)
    return result + [0] * (SIZE - len(result))
```

It always compares the next value with `result[-1]`, even when that tile was just made by a merge, so $2+2$ makes 4 and that 4 then merges straight away with the next 4. There is also no line that adds to the score, and because it was a `@staticmethod` it couldn't reach `self.score` anyway.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\apurv\College\sem 5\Software Engineering\PES1UG24CS077_Repo_Upload\14_2048> python -c "from board import Board; b = Board(); b.slide_line([2,2,4,0]); print(b.board)"
[[8, 0, 0, 0]]
PS C:\Users\apurv\College\sem 5\Software Engineering\PES1UG24CS077_Repo_Upload\14_2048> python main.py
2048 - W/A/S/D to move, U to undo, Q to quit.

+-----+
|       |       |       |       |
+-----+
|       |  2  |       |  2  |
+-----+
|       |       |       |       |
+-----+
|       |       |       |       |
+-----+
Score: 0 Best: 0
> a

+-----+
|  4  |       |       |       |
+-----+
|       |       |       |       |
+-----+
|       |       |  2  |       |
+-----+
Score: 0 Best: 0
>
```

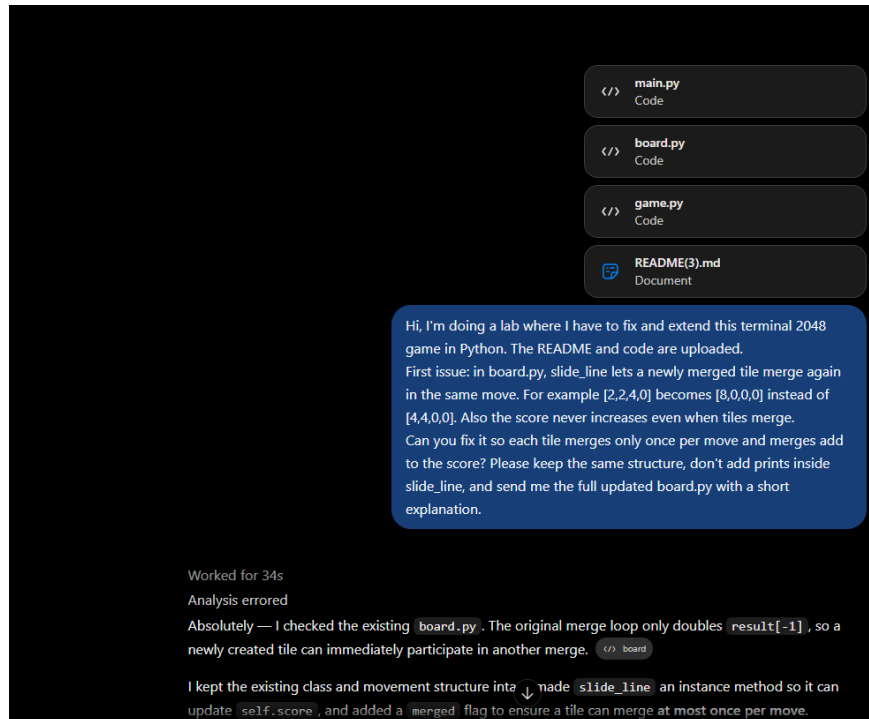
From my before video: `slide_line([2,2,4,0])` prints `[8, 0, 0, 0]`, and the two 2s merge into a 4 but the score stays at 0.

3. Task 1 – Merging correctly and keeping score

Prompt 1 of 4 · commit dfb760d

The prompt

I started with the most obvious bug. I gave ChatGPT a concrete failing example and asked it to keep the existing structure and not add any prints, because I didn't want it to rewrite the whole file.



Screenshot of prompt 1 in ChatGPT.

What ChatGPT changed

ChatGPT spotted the problem straight away. It made `slide_line` an instance method and added a `merged` flag, so a tile that has just merged can't merge again in the same move. It also adds the new tile's value to `self.score`. The four move functions now call `self.slide_line(...)`. Nothing else changed.

`board.py` – `slide_line` after the fix:

```
def slide_line(self, line):
    values = [x for x in line if x]
    result = []
    merged = False

    for value in values:
        if result and result[-1] == value and not merged:
            result[-1] *= 2
            self.score += result[-1]
            merged = True
        else:
            result.append(value)
            merged = False

    return result + [0] * (SIZE - len(result))
```

How I tested it

To test it, I called `slide_line` on the cases from the README and then played a move in the real game. `[4, 4, 8, 0]` now gives `[8, 8, 0, 0]`, `[2, 2, 2, 2]` gives `[4, 4, 0, 0]`, and `[2, 4, 2, 4]` stays the same. When I pressed D the two 2s merged and the score went from 0 to 4.

```
Windows PowerShell
[4, 4, 8, 0] -> [8, 8, 0, 0]
[2, 2, 2, 2] -> [4, 4, 0, 0]
[2, 4, 2, 4] -> [2, 4, 2, 4]
PS C:\Users\apurv\College\sem 5\Software Engineering\PES1UG24CS077_Repo_Upload\Micro-Lending-Peer-Credit-Risk-Assessor\Lab-4> python main.py
2048 - W/A/S/D to move, U to undo, Q to quit.

+-----+-----+-----+-----+
|         |         |         |         |
|         |         |         |         |
|         |         |         |         |
|         |         |         |         |
|         |         |         |         |
|         |         |         |         |
+-----+-----+-----+-----+
Score: 0 Best: 0
> d
Moved right: 1 merges, +4 points

+-----+-----+-----+-----+
|         |         |         |         |
|         |         |         |         |
|         |         |         |         |
|         |         |         |         |
|         |         |         |         |
|         |         |         |         |
+-----+-----+-----+-----+
Score: 4 Best: 4
> |
```

Task 1 working: correct merge results, and the score goes up to 4 after a merge.

Before vs after, side by side

To show the difference clearly, I ran the same short script on the code just before this commit and on the code after it. This is real program output, with the board drawing removed to save space.

Before (249cf2d)

```
>>> slide_line([2, 2, 4, 0]) -> [8, 0, 0, 0] score += 0
>>> slide_line([4, 4, 8, 0]) -> [16, 0, 0, 0] score += 0
>>> slide_line([2, 2, 2, 2]) -> [4, 4, 0, 0] score += 0
>>> slide_line([2, 0, 2, 2]) -> [4, 2, 0, 0] score += 0
```

After (dfb760d)

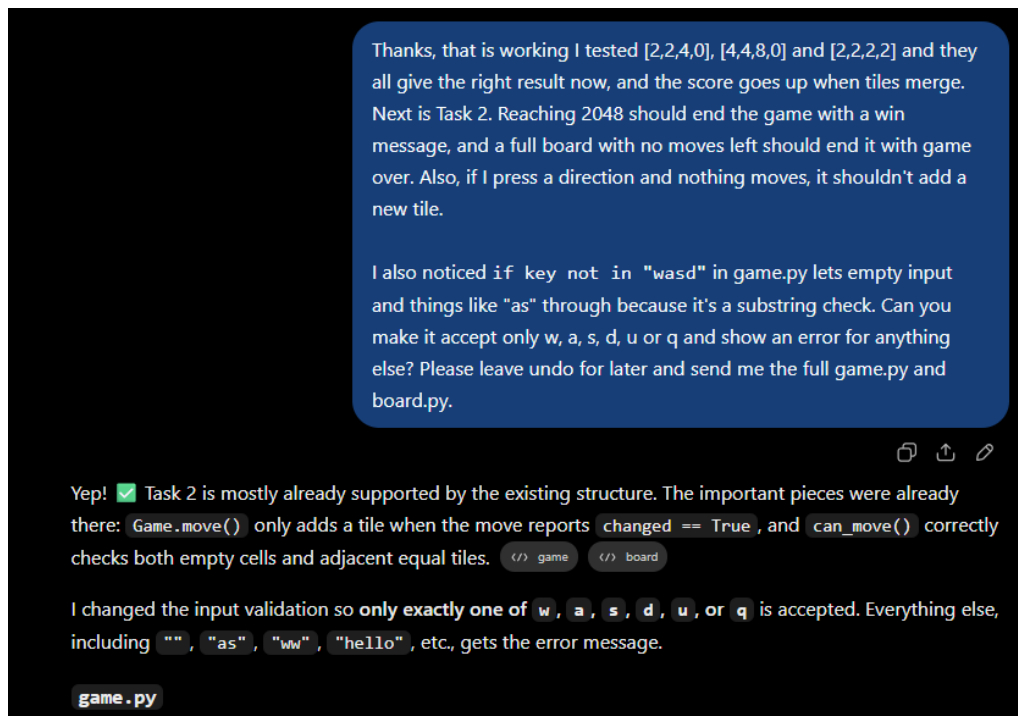
```
>>> slide_line([2, 2, 4, 0]) -> [4, 4, 0, 0] score += 4
>>> slide_line([4, 4, 8, 0]) -> [8, 8, 0, 0] score += 8
>>> slide_line([2, 2, 2, 2]) -> [4, 4, 0, 0] score += 8
>>> slide_line([2, 0, 2, 2]) -> [4, 2, 0, 0] score += 4
```

4. Task 2 – Win, game over, unchanged moves and input checking

Prompt 2 of 4 · commit 3bf024a

The prompt

For Task 2 I explained the game-state rules from the README and also pointed out the "wasd" substring problem I had noticed myself. I told it to leave undo alone for now, so each commit would only cover one task.



Screenshot of prompt 2 in ChatGPT.

What ChatGPT changed

ChatGPT pointed out that most of Task 2 was already there: the 2048 check and `can_move()` worked, and `Game.move()` only added a tile when `changed` was true. I checked this myself and it was correct. The real fix was the input check: it now uses an exact set `{"w", "a", "s", "d"}` with a clear error message, and the game-over message now actually says "Game over!".

`game.py` – input check after the fix:

```
if key not in {"w", "a", "s", "d"}:
    print("Use W/A/S/D, U to undo, or Q to quit.")
    continue
```

game-over message:

```
if not self.board.can_move():
    print("No legal moves remain. Game over!")
    return
```

How I tested it

I typed "as" and "xyz" and both showed the error without changing the board. Pressing W when nothing could move up printed "No tiles moved." and didn't add a tile. To test the win without playing a whole game, I set up a board with two 1024 tiles and moved left: they merged into 2048 and the win

check returned True. A full checkerboard of 2s and 4s gave `can_move() = False`, which is the game-over case.

```

Windows PowerShell
PS C:\Users\apurv\College\sem 5\Software Engineering\PES1UG24CS077_Repo_Upload\Micro-Lending-Peer-Credit-Risk-Assessor\Lab-4> python main.py
2048 - W/A/S/D to move, U to undo, Q to quit.

+-----+
| 2     | 2     |
+-----+
|       |       |
+-----+
|       |       |
+-----+
|       |       |
+-----+
Score: 0 Best: 0
> as
Use W/A/S/D, U to undo, or Q to quit.

+-----+
| 2     | 2     |
+-----+
|       |       |
+-----+
|       |       |
+-----+
|       |       |
+-----+
Score: 0 Best: 0
> xyz
Use W/A/S/D, U to undo, or Q to quit.

+-----+
| 2     | 2     |
+-----+
|       |       |
+-----+
|       |       |
+-----+
|       |       |
+-----+
Score: 0 Best: 0
> W
No tiles moved.

+-----+
| 2     | 2     |
+-----+
|       |       |
+-----+
|       |       |
+-----+
|       |       |
+-----+
Score: 0 Best: 0
>

```

Invalid input ("as", "xyz") is rejected, and an unchanged move adds no tile.

```

Windows PowerShell
PS C:\Users\apurv\College\sem 5\Software Engineering\PES1UG24CS077_Repo_Upload\Micro-Lending-Peer-Credit-Risk-Assessor\Lab-4> python -c "from game import Game; g=Game(); g.board.grid=[[1024,1024,0,0],[0]*4,[0]*4,[0]*4]; g.move('a'); g.display(); print('Win:', any(2048 in r for r in g.board.grid))"
Moved left: 1 merges, +2048 points

+-----+
| 2048  |       |       |       |
+-----+
|       |       |       |       |
+-----+
|       |       |       |       |
+-----+
|       |       |       | 2     |
+-----+
Score: 2048 Best: 2048
Win: True
PS C:\Users\apurv\College\sem 5\Software Engineering\PES1UG24CS077_Repo_Upload\Micro-Lending-Peer-Credit-Risk-Assessor\Lab-4> python -c "from board import Board; b=Board(); b.grid=[[2,4,2,4],[4,2,4,2],[2,4,2,4],[4,2,4,2]]; print('Can move:', b.can_move())"
Can move: False
PS C:\Users\apurv\College\sem 5\Software Engineering\PES1UG24CS077_Repo_Upload\Micro-Lending-Peer-Credit-Risk-Assessor\Lab-4>

```

Win: $1024 + 1024 = 2048$. Full board with no moves left: `can_move()` is False.

Before vs after, side by side

To show the difference clearly, I ran the same short script on the code just before this commit and on the code after it. This is real program output, with the board drawing removed to save space.

Before (249cf2d)

```
# invalid input check
2048 - W/A/S/D to move, U to undo, Q to quit.

Score: 0 Best: 0
>
Score: 0 Best: 0
> as
Score: 0 Best: 0
> xyz
Use W/A/S/D.

Score: 0 Best: 0
[end of scripted input]

# unchanged move adds no tile
>>> move('a') changed=False, grid unchanged=True

# win at 2048
2048 - W/A/S/D to move, U to undo, Q to quit.

Score: 0 Best: 0
You reached 2048!

# full board, no moves
2048 - W/A/S/D to move, U to undo, Q to quit.

Score: 0 Best: 0
No legal moves remain.
```

After (3bf024a)

```
# invalid input check
2048 - W/A/S/D to move, U to undo, Q to quit.

Score: 0 Best: 0
>
Use W/A/S/D, U to undo, or Q to quit.

Score: 0 Best: 0
> as
Use W/A/S/D, U to undo, or Q to quit.

Score: 0 Best: 0
> xyz
Use W/A/S/D, U to undo, or Q to quit.

Score: 0 Best: 0
[end of scripted input]

# unchanged move adds no tile
>>> move('a') changed=False, grid unchanged=True

# win at 2048
2048 - W/A/S/D to move, U to undo, Q to quit.

Score: 0 Best: 0
You reached 2048!

# full board, no moves
2048 - W/A/S/D to move, U to undo, Q to quit.

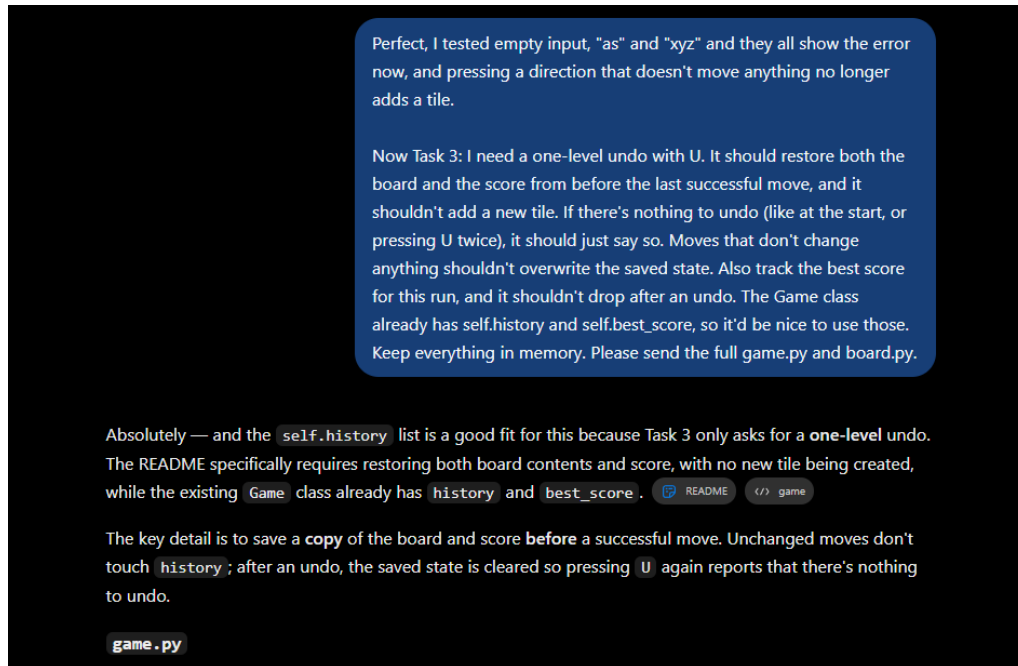
Score: 0 Best: 0
No legal moves remain. Game over!
```


5. Task 3 – One-level undo and best score

Prompt 3 of 4 · commit 5586c48

The prompt

For undo I listed the edge cases I could think of: undo straight after starting, pressing U twice, and a move that does nothing wiping out the saved state. I also said to use the `self.history` and `self.best_score` that were already in the class.



Screenshot of prompt 3 in ChatGPT.

What ChatGPT changed

`Game.move()` now saves a copy of the grid and the score before moving, but only writes it to `history` if the move actually changed something. `undo()` takes that state back out, restores the board and the score, and doesn't add a tile. Pressing U a second time gives "Nothing to undo.". Best score only ever goes up through `max()`, so an undo can't lower it.

`game.py` – saving state and undo:

```

def move(self, key):
    moves = {
        "a": self.board.move_left,
        "d": self.board.move_right,
        "w": self.board.move_up,
        "s": self.board.move_down
    }

    if key not in moves:
        return False

    old_grid = [row[:] for row in self.board.grid]
    old_score = self.board.score

    changed = moves[key]()

    if changed:
        self.history = [(old_grid, old_score)]
        self.board.add_random_tile()

    return changed

def undo(self):
    if not self.history:
        print("Nothing to undo.")
        return False

    old_grid, old_score = self.history.pop()
    self.board.grid = [row[:] for row in old_grid]
    self.board.score = old_score

    print("Move undone.")
    return True

```

How I tested it

I merged two 2s (score 4, best 4), pressed U and the board went back exactly as it was, with the score at 0. Best score stayed at 4, which is what I wanted. Pressing U again printed "Nothing to undo."

```
Windows PowerShell
PS C:\Users\apurv\College\sem 5\Software Engineering\PES1UG24CS077_Repo_Upload\Micro-Lending-Peer-Credit-Risk-Assessor\Lab-4> python main.py
2048 - W/A/S/D to move, U to undo, Q to quit.

+-----+
|       |       |       |       |
|       |       |       |       |
|       |  2  |       |       |
|       |  2  |       |       |
|       |       |       |       |
+-----+
Score: 0 Best: 0
> s
Moved down: 1 merges, +4 points

+-----+
|  2  |       |       |       |
|       |       |       |       |
|       |       |       |       |
|       |  4  |       |       |
|       |       |       |       |
+-----+
Score: 4 Best: 4
> u
Move undone.

+-----+
|       |       |       |       |
|       |  2  |       |       |
|       |  2  |       |       |
|       |       |       |       |
|       |       |       |       |
+-----+
Score: 0 Best: 4
> u
Nothing to undo.

+-----+
|       |       |       |       |
|       |  2  |       |       |
|       |  2  |       |       |
|       |       |       |       |
|       |       |       |       |
+-----+
Score: 0 Best: 4
> |
```

Undo puts back the board and the score, best score stays at 4, and a second U says "Nothing to undo."

Before vs after, side by side

To show the difference clearly, I ran the same short script on the code just before this commit and on the code after it. This is real program output, with the board drawing removed to save space.

Before (249cf2d)

```
2048 - W/A/S/D to move, U to undo, Q to quit.

Score: 0 Best: 0
> a

Score: 0 Best: 0
> u
Undo is not implemented yet.

Score: 0 Best: 0
> u
Undo is not implemented yet.

Score: 0 Best: 0
[end of scripted input]
```

After (5586c48)

```
2048 - W/A/S/D to move, U to undo, Q to quit.

Score: 0 Best: 0
> a

Score: 4 Best: 4
> u
Move undone.

Score: 0 Best: 4
> u
Nothing to undo.

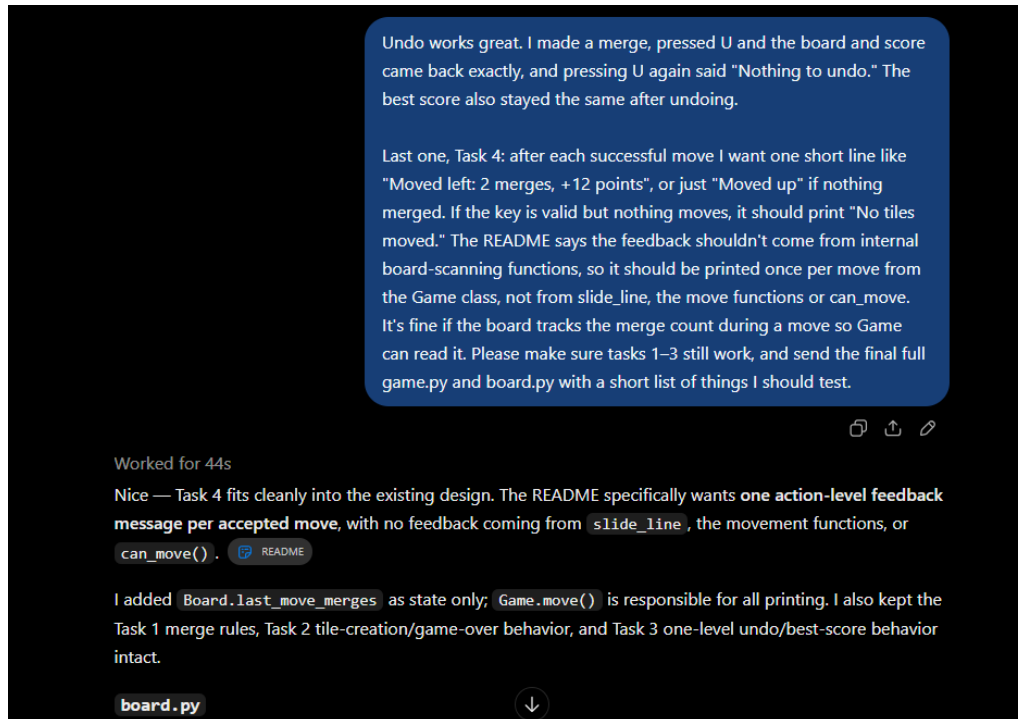
Score: 0 Best: 4
[end of scripted input]
```

6. Task 4 – Feedback after each move

Prompt 4 of 4 · commit 33dfc00

The prompt

For the last prompt I gave the exact messages I wanted and repeated the README rule that feedback must come from the Game class only, not from `slide_line`, the move functions or `can_move`. I also asked it to make sure Tasks 1–3 still worked and to give me a list of things to test.



Screenshot of prompt 4 in ChatGPT.

What ChatGPT changed

The board now has a `last_move_merges` counter. Each move function resets it to 0, and `slide_line` adds 1 for every merge (it only stores the number, it doesn't print). `Game.move()` reads the counter and the change in score, then prints exactly one line. Because `can_move()` never prints, there are no repeated messages when the board is checked.

game.py – feedback in `Game.move`:

```
points = self.board.score - old_score
...
if self.board.last_move_merges:
    print(f"Moved {direction}: "
          f"{self.board.last_move_merges} merges, "
          f"{points} points")
else:
    print(f"Moved {direction}")
...
else:
    print("No tiles moved.")
```

How I tested it

I played a few moves. Moving left with no merge printed "Moved left", pressing A again when nothing could move printed "No tiles moved.", and moving up to merge two 2s printed "Moved up: 1 merges, +4 points". I also ran through the Task 1–3 checks again and they all still worked. One small thing:

with a single merge it says "1 merges" instead of "1 merge". It's only cosmetic, so I left it.

```
Windows PowerShell
PS C:\Users\apurv\College\sem 5\Software Engineering\PES1UG24CS077_Repo_Upload\Micro-Lending-Peer-Credit-Risk-Assessor\Lab-4> python main.py
2048 - W/A/S/D to move, U to undo, Q to quit.

+-----+
|       |       |       |       |
|  2     |       |       |       |
+-----+
|       |       |       |  2     |
+-----+
|       |       |       |       |
+-----+
Score: 0  Best: 0
> a
Moved left

+-----+
|  2     |       |       |       |
|  2     |       |       |       |
+-----+
|  2     |       |       |       |
+-----+
Score: 0  Best: 0
> a
No tiles moved.

+-----+
|  2     |       |       |       |
|  2     |       |       |       |
+-----+
|  2     |       |       |       |
+-----+
Score: 0  Best: 0
> w
Moved up: 1 merges, +4 points

+-----+
|  4     |       |  2     |       |
|  2     |       |       |       |
+-----+
|       |       |       |       |
+-----+
Score: 4  Best: 4
> |
```

One feedback line per move: "Moved left", "No tiles moved." and "Moved up: 1 merges, +4 points".

Before vs after, side by side

To show the difference clearly, I ran the same short script on the code just before this commit and on the code after it. This is real program output, with the board drawing removed to save space.

Before (5586c48)

```
2048 - W/A/S/D to move, U to undo, Q to quit.

Score: 0  Best: 0
> a

Score: 12  Best: 12
> a

Score: 12  Best: 12
> w

Score: 12  Best: 12
[end of scripted input]
```

After (33dfc00)

```
2048 - W/A/S/D to move, U to undo, Q to quit.

Score: 0  Best: 0
> a
Moved left: 2 merges, +12 points

Score: 12  Best: 12
> a
Moved left

Score: 12  Best: 12
> w
No tiles moved.

Score: 12  Best: 12
[end of scripted input]
```

7. Testing summary

This is everything I checked on the final version (commit 33dfc00). Some of it I did by playing the game (screenshots above) and some with quick one-line Python commands. Whenever a later task touched the same code, I ran the earlier checks again.

#	Test	Expected	Result
1	slide_line([2,2,4,0])	[4,4,0,0], +4	Pass
2	slide_line([4,4,8,0])	[8,8,0,0], +8	Pass
3	slide_line([2,2,2,2])	[4,4,0,0], +8	Pass
4	slide_line([2,4,2,4])	Unchanged	Pass
5	Score on merge	Goes up by the merged value	Pass
6	Reach 2048	"You reached 2048!", game ends	Pass
7	Full board, no moves left	can_move() False, "Game over!"	Pass
8	Direction that moves nothing	No new tile, "No tiles moved."	Pass
9	Empty input / "as" / "xyz"	Error message, board unchanged	Pass
10	U after a move	Board and score restored, no new tile	Pass
11	U at start / U twice	"Nothing to undo."	Pass
12	Unchanged move after a real move	Saved undo state not overwritten	Pass
13	Best score after undo	Doesn't drop	Pass
14	Move with merges	"Moved up: 1 merges, +4 points"	Pass
15	Move without merges	"Moved left"	Pass
16	can_move() / move functions	Print nothing	Pass
17	Merge counter	Reset at the start of every move	Pass
18	Q	Quits cleanly	Pass
19	Tasks 1–3 again on final code	Still work	Pass

8. Commit history

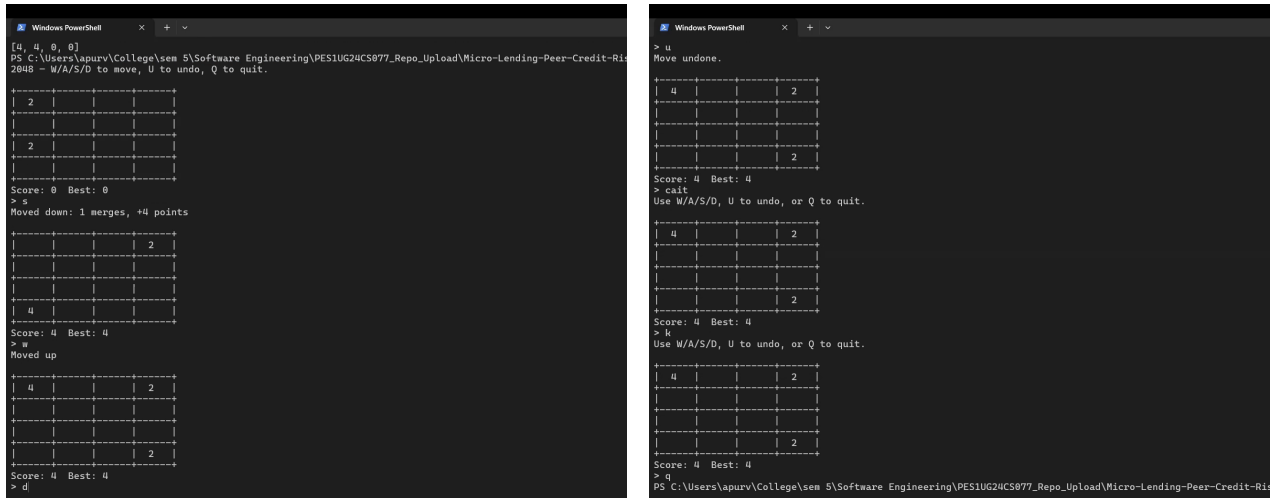
Each task has its own commit, and the starter code and the before video went in first so the history shows the progress step by step.

```
Windows PowerShell
PS C:\Users\apurv\College\sem 5\Software Engineering\PES1UG24CS077_Repo_Upload\Micro-Lending-Peer-Credit-Risk-Assessor> git log --oneline -10
e155901 (HEAD -> main, origin/main, origin/HEAD) Update README with Lab 4
7dd744d Lab-4: after video and LLM chat history
33dfc00 Lab-4 Task 4: action-level move feedback
5586c48 Lab-4 Task 3: one-level undo and best score
3bf024a Lab-4 Task 2: win/no-move detection, no tile on unchanged move, input validation
dfb760d Lab-4 Task 1: fix double-merge bug and score update
ae49c00 Lab-4: before video showing double-merge bug
249cf2d Lab-4: add 14_2048 starter code (unmodified)
fd26f4c Reorganise into lab folders, fix formatting, add Lab 2 deliverables to repo
f14eb94 Add files via upload
PS C:\Users\apurv\College\sem 5\Software Engineering\PES1UG24CS077_Repo_Upload\Micro-Lending-Peer-Credit-Risk-Assessor>
```

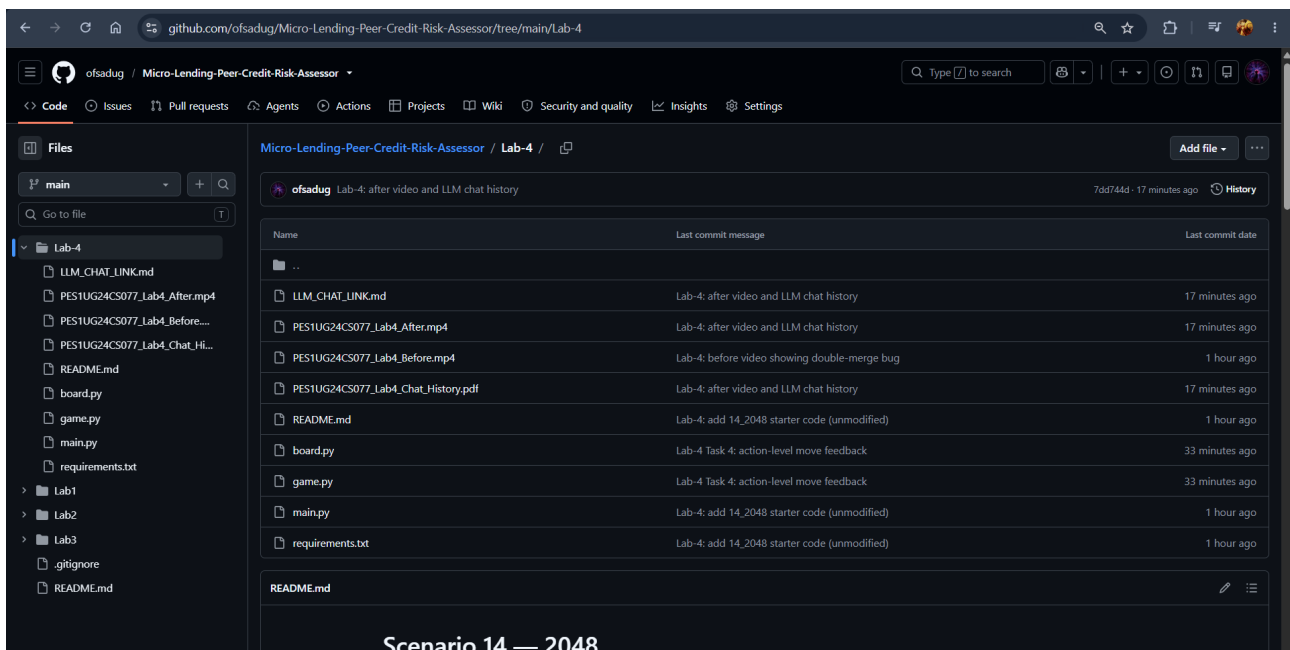
git log --oneline in my repository.

9. Videos and the repository

Before video ([PES1UG24CS077_Lab4_Before.mp4](#), about 11 s): shows the double merge ([8, 0, 0, 0]) and the score not going up. **After video** ([PES1UG24CS077_Lab4_After.mp4](#)): the same check now gives [4, 4, 0, 0], and it also shows the score, the feedback lines, undo and invalid input.



Two frames from the after video.



The Lab-4 folder on GitHub with the code, both videos, the chat history PDF and the chat link file.

10. Links

Item	Link
Repository	https://github.com/ofsadug/Micro-Lending-Peer-Credit-Risk-Assessor
Lab-4 folder	https://github.com/ofsadug/Micro-Lending-Peer-Credit-Risk-Assessor/tree/main/Lab-4
Before video	https://github.com/ofsadug/Micro-Lending-Peer-Credit-Risk-Assessor/blob/main/Lab-4/PES1UG24CS077_Lab4_Before.mp4
After video	https://github.com/ofsadug/Micro-Lending-Peer-Credit-Risk-Assessor/blob/main/Lab-4/PES1UG24CS077_Lab4_After.mp4

Item	Link
ChatGPT chat	https://chatgpt.com/share/6ac65eff-d0ac-83ec-85d4-1b1cf8b0613b
Chat history PDF	https://github.com/ofsadug/Micro-Lending-Peer-Credit-Risk-Assessor/blob/main/Lab-4/PES1UG24CS077_Lab4_Chat_History.pdf

11. What I took away from this

The biggest lesson was that how good the AI's answer is depends a lot on how specific I am. When I gave it an exact failing input like `[2, 2, 4, 0] → [8, 0, 0, 0]`, it found the bug right away. Saying what not to touch (keep the structure, no prints in `slide_line`, feedback only from Game) kept the changes small, which made each commit easy to check.

I also learned not to just trust the output. In Task 2, ChatGPT said part of the task was already done, and I only believed it after testing it myself. Checking every reply before committing took a bit longer, but it meant I always knew each commit actually worked. With only 4 prompts, starting each one by telling it what I had already tested also helped it stay on track.

If I had one more prompt I would fix the "1 merges" wording, but it doesn't affect how the game works, so I left it as a known issue.